

INFORMAÇÕES		
MÓDULO	AULA	INSTRUTOR
09	Introdução ao React para desenvolvedores .NET	03 - Introdução ao React com Vite + TypeScript Pedro Henrique Amadio

CONTEÚDO
1. O que é React? O React é uma biblioteca JavaScript usada para construir interfaces de usuário. A documentação oficial organiza React em torno de componentes, Hooks e APIs usadas para montar interfaces. Quando falamos em interface, estamos falando da parte visual da aplicação: • títulos; • botões; • formulários; • listas; • menus; • cards; • telas em geral. A grande ideia do React é permitir que a interface seja construída em partes menores, reutilizáveis e organizadas. Essas partes recebem o nome de componentes. Em vez de criar uma página inteira como um bloco único e grande, o React incentiva que a aplicação seja dividida em pequenas unidades visuais e lógicas. Isso ajuda muito em: • organização; • reaproveitamento; • manutenção; • clareza do código; • crescimento do projeto.
2. O que é TSX? Como esta aula será feita em React com TypeScript, os arquivos principais da interface serão escritos em TSX. Esse é um ponto que vale explicar com bastante clareza: • JSX é a sintaxe visual parecida com HTML usada dentro do React; • TSX é o arquivo TypeScript que contém JSX.

Ou seja:

- em React com JavaScript, usamos `.jsx`;
- em React com TypeScript, usamos `.tsx`.

Nesta aula, trabalharemos principalmente com:

- `main.tsx`
- `App.tsx`

3. O que é JSX?

JSX é a sintaxe usada pelo React para descrever a interface.

Ela se parece com HTML, mas está sendo usada dentro do JavaScript ou TypeScript.

Exemplo:

```
function App() {  
  return <h1>Minha aplicação React</h1>;  
}
```

Isso parece HTML, mas faz parte do código do componente.

O JSX facilita muito a leitura, porque deixa a interface mais próxima daquilo que enxergamos visualmente.

Exemplo com mais elementos:

```
function App() {  
  return (  
    <div>  
      <h1>Lista de Tarefas</h1>  
      <p>Primeiro contato com React + TypeScript</p>  
    </div>  
  );  
}
```

Regra importante

O componente precisa retornar um único elemento pai.

Por isso, normalmente usamos:

- uma `<div>`;
- ou um fragmento `<> </>`.

Exemplo correto:

```
function App() {  
  return (  

```

```
<div>
  <h1>Título</h1>
  <p>Texto</p>
</div>
);
}
```

4. O que é uma SPA e onde o React se encaixa nisso?

SPA significa **Single Page Application**, ou **Aplicação de Página Única**.

Isso não quer dizer que a aplicação terá apenas uma única tela visual, mas sim que ela funciona com uma base principal carregada no navegador e vai atualizando a interface dinamicamente sem recarregar a página inteira a cada ação.

Em aplicações tradicionais, navegar pode significar carregar um novo HTML completo. Em aplicações React, a interface costuma ser atualizada de forma mais fluida, reaproveitando a estrutura já carregada.

Esse é um dos motivos pelos quais React é muito usado em front-end moderno.

5. Componentes: a base do React

Componentes são blocos reutilizáveis de interface.

Um componente pode representar:

- um cabeçalho;
- um botão;
- uma área de formulário;
- um item da lista;
- uma seção da tela.

Em código moderno, o React recomenda o uso de componentes de função em vez de componentes de classe.

Exemplo simples:

```
function Titulo() {
  return <h1>Olá, React!</h1>;
}
```

Nesse caso:

- Titulo é um componente;
- ele é uma função;
- ele retorna algo visual para a interface.

6. JSX + lógica juntos

No React, podemos misturar lógica com interface de forma organizada. Usamos chaves { } para inserir expressões JavaScript no JSX.

Exemplo:

```
function App() {
  const nome: string = "Pedro";

  return (
    <div>
      <h1>Olá, {nome}!</h1>
    </div>
  );
}
```

Também podemos usar arrays:

```
function App() {
  const tecnologias: string[] = ["HTML", "CSS", "TypeScript", "React"];

  return (
    <ul>
      {tecnologias.map((item) => (
        <li key={item}>{item}</li>
      ))}
    </ul>
  );
}
```

7. O que são Hooks?

Hooks são recursos do React usados dentro de componentes para adicionar funcionalidades como estado e efeitos. A documentação oficial lista `useState` entre os Hooks de estado e `useEffect` como o Hook usado para sincronização com sistemas externos.

Nesta aula, vamos focar em dois nomes muito importantes:

- `useState`
- `useEffect`

8. `useState` - guardando valores que mudam

A documentação oficial define `useState` como um Hook que permite adicionar uma variável de estado ao componente.

Estado é tudo aquilo que pode mudar durante o uso da aplicação.

Exemplos:

- o texto digitado no input;
- a lista de tarefas;
- um contador;
- um item selecionado;
- um menu aberto ou fechado.

Exemplo simples:

```
import { useState } from "react";

function App() {
  const [contador, setContador] = useState<number>(0);
```

```

    return (
      <div>
        <p>Valor atual: {contador}</p>
        <button onClick={() => setContador(contador + 1)}>Incrementar</button>
      </div>
    );
  }
}
export default App;

```

Entendendo a linha principal

```
const [contador, setContador] = useState<number>(0);
```

Aqui temos:

- contador: valor atual;
- setContador: função que atualiza o valor;
- 0: valor inicial;
- <number>: tipo do estado.

9. useEffect - primeira visão

A documentação do React define useEffect como um Hook que permite sincronizar um componente com um sistema externo. Ela também explica, em outro material oficial, que nem toda lógica precisa de Effect; quando não existe sistema externo envolvido, muitas vezes ele não é necessário.

Isso significa que o useEffect é útil para situações como:

- buscar dados;
- salvar no localStorage;
- alterar o título da página;
- interagir com algo externo ao fluxo normal da renderização.

Exemplo inicial:

```

import { useEffect } from "react";

function App() {
  useEffect(() => {
    console.log("Componente renderizado");
  }, []);

  return <h1>Olá!</h1>;
}

```

10. Criando o projeto com React + TypeScript + Vite

10.1. Pré-requisito

Verifique se o Node.js instalado é compatível com a versão atual do Vite. O guia atual informa Node.js **20.19+** ou **22.12+**.

10.2. Comando para criar o projeto

No CMD:

```
npm create vite@latest react-ts-vite-app -- --template react-ts
```

Entrando na pasta do projeto

```
cd react-ts-vite-app
```

Instalando dependências

```
npm install
```

Executando o projeto

```
npm run dev
```

Depois disso, o Vite exibirá um endereço local no terminal para abrir no navegador.

10.3. Estrutura inicial do projeto

Depois que o projeto estiver criado, apresente a estrutura de forma simples.

Arquivos principais:

- src/main.tsx
- src/App.tsx
- src/index.css
- package.json

O papel de cada um

src/main.tsx

É o ponto de entrada da aplicação.

src/App.tsx

É o componente principal que será exibido.

src/index.css

Arquivo de estilos globais iniciais.

package.json

Arquivo com dependências e scripts do projeto.

Nesta aula, vamos trabalhar principalmente em:

- src/App.tsx

10.4. Limpando o projeto inicial

O projeto criado pelo Vite vem com uma tela padrão de exemplo.

Para a aula ficar mais clara, vamos limpar o conteúdo do App.tsx e começar com algo simples.

Exemplo inicial

```
function App() {
  return (
    <div>
      <h1>Lista de Tarefas</h1>
      <p>Minha primeira aplicação com React + TypeScript</p>
    </div>
  );
}

export default App;
```

10.5. Criando os estados da aplicação

Vamos importar o useState e criar dois estados:

- um para o texto atual digitado;
- outro para a lista de tarefas.

```
import { useState } from "react";

function App() {
  const [novaTarefa, setNovaTarefa] = useState<string>("");
  const [tarefas, setTarefas] = useState<string[]>([]);

  return (
    <div>
      <h1>Lista de Tarefas</h1>
    </div>
  );
}

export default App;
```

Explicando as tipagens

```
const [novaTarefa, setNovaTarefa] = useState<string>("");
```

1. novaTarefa será uma string;
2. começa vazia;
3. será atualizada conforme o usuário digita.

```
const [tarefas, setTarefas] = useState<string[]>([]);
```

- tarefas será um array de strings;
- começa vazio;
- crescerá conforme adicionarmos novas tarefas.

10.6. Criando o input e o botão

Agora vamos inserir o campo de texto e o botão.

```
import { ChangeEvent, useState } from "react";

function App() {
  const [novaTarefa, setNovaTarefa] = useState<string>("");
  const [tarefas, setTarefas] = useState<string[]>([]);

  function handleChange(event: ChangeEvent<HTMLInputElement>): void {
    setNovaTarefa(event.target.value);
  }
}
```

```

return (
  <div>
    <h1>Lista de Tarefas</h1>

    <input
      type="text"
      placeholder="Digite uma tarefa"
      value={novaTarefa}
      onChange={handleChange}
    />

    <button>Adicionar</button>
  </div>
);
}

export default App;

```

Nesse trecho, temos três pontos importantes:

1. Input controlado

O valor do input está ligado ao estado novaTarefa.

2. Evento tipado

O TypeScript permite tipar o evento:

```
ChangeEvent<HTMLInputElement>
```

3. Função com retorno void

Como a função não retorna valor, usamos : void.

10.7. Fazendo o botão adicionar funcionar

Agora vamos criar a função que adiciona a tarefa na lista.

```

import { ChangeEvent, useState } from "react";

function App() {
  const [novaTarefa, setNovaTarefa] = useState<string>("");
  const [tarefas, setTarefas] = useState<string[]>([]);

  function handleChange(event: ChangeEvent<HTMLInputElement>): void {
    setNovaTarefa(event.target.value);
  }

  function adicionarTarefa(): void {
    if (novaTarefa.trim() === "") {
      return;
    }

    setTarefas([...tarefas, novaTarefa]);
    setNovaTarefa("");
  }

  return (
    <div>
      <h1>Lista de Tarefas</h1>

      <input
        type="text"
        placeholder="Digite uma tarefa"

```

```

        value={novaTarefa}
        onChange={handleChange}
      />

      <button onClick={adicionarTarefa}>Adicionar</button>
    </div>
  );
}

export default App;

```

O que está acontecendo aqui?

Primeiro:

```

if (novaTarefa.trim() === "") {
  return;
}

```

Isso impede que uma tarefa vazia seja adicionada.

Depois:

```

setTarefas([...tarefas, novaTarefa]);

```

Criamos um novo array contendo:

- todas as tarefas antigas;
- a nova tarefa ao final.

Por fim:

```

setNovaTarefa("");

```

Limpamos o campo de texto.

10.8. Renderizando a lista na tela

Agora vamos exibir as tarefas adicionadas.

```

import { ChangeEvent, useState } from "react";

function App() {
  const [novaTarefa, setNovaTarefa] = useState<string>("");
  const [tarefas, setTarefas] = useState<string[]>([]);

  function handleChange(event: ChangeEvent<HTMLInputElement>): void {
    setNovaTarefa(event.target.value);
  }

  function adicionarTarefa(): void {
    if (novaTarefa.trim() === "") {
      return;
    }

    setTarefas([...tarefas, novaTarefa]);
    setNovaTarefa("");
  }

  return (
    <div>
      <h1>Lista de Tarefas</h1>
      <p>Minha primeira aplicação com React + TypeScript</p>
    </div>
  );
}

```

```

    <input
      type="text"
      placeholder="Digite uma tarefa"
      value={novaTarefa}
      onChange={handleChange}
    />

    <button onClick={adicionarTarefa}>Adicionar</button>

    <ul>
      {tarefas.map((tarefa, indice) => (
        <li key={indice}>{tarefa}</li>
      ))}
    </ul>
  </div>
);
}

export default App;

```

Note que existe uma propriedade nova “key” e ela é responsável por ser a chave única do item a ser renderizado, portanto KEY nunca deve se repetir dentro da página. Uma boa prática é usar prefixos e sufixos junto com o index(índice) do item: `tarefa-{indice}-item`

10.9. Versão final da atividade prática da aula

Aqui está a versão principal da atividade que pode ser construída junto com a turma:

```

import { ChangeEvent, useState } from "react";

function App() {
  const [novaTarefa, setNovaTarefa] = useState<string>("");
  const [tarefas, setTarefas] = useState<string[]>([]);

  function handleChange(event: ChangeEvent<HTMLInputElement>): void {
    setNovaTarefa(event.target.value);
  }

  function adicionarTarefa(): void {
    if (novaTarefa.trim() === "") {
      return;
    }

    setTarefas([...tarefas, novaTarefa]);
    setNovaTarefa("");
  }

  return (
    <div>
      <h1>Lista de Tarefas</h1>
      <p>Minha primeira aplicação com React + TypeScript</p>

      <input
        type="text"
        placeholder="Digite uma tarefa"
        value={novaTarefa}
        onChange={handleChange}

```

```

    />

    <button onClick={adicionarTarefa}>Adicionar</button>

    <ul>
      {tarefas.map((tarefa, indice) => (
        <li key={indice}>{tarefa}</li>
      ))}
    </ul>
  </div>
);
}

export default App;

```

11. Para casa

Atualize o exercício prático da aula de hoje e adicione uma exclusão para cada tarefa

Regras

- não permitir tarefa vazia;
- limpar o campo após adicionar;
- mostrar todas as tarefas na tela;
- usar useState com tipagem adequada.

12. Exercícios práticos

1. Em um projeto React com TypeScript, qual extensão usamos normalmente nos componentes que possuem JSX?

- ☐ .js
- ☐ .jsx
- ☐ .tsx
- ☐ .json

2. O que é um componente em React?

- ☐ Um bloco reutilizável de interface
- ☐ Um tipo de CSS automático
- ☐ Um arquivo obrigatório do navegador
- ☐ Um plugin do Vite

3. Para que serve o useState?

- ☐ Instalar dependências no projeto
- ☐ Criar rotas automáticas
- ☐ Armazenar e atualizar valores que mudam no componente
- ☐ Gerar HTML estático

4. O comando abaixo faz o quê?

```
npm create vite@latest react-ts-vite-app -- --template react-ts
```

- ☐ Executa o build final da aplicação
- ☐ Cria um projeto React + TypeScript com Vite
- ☐ Remove dependências antigas
- ☐ Publica o projeto na internet

5. Qual das opções abaixo representa corretamente um estado tipado de lista de textos?

- ☐ `useState<number>([])`
- ☐ `useState<string[]>([])`
- ☐ `useState<boolean>("")`
- ☐ `useState<object>("tarefa")`